Nama : Khania Puji Auliya

NIM : 254107020236

Kelas : 1G

Prodi : D-IV Teknik Informatika

LAPORAN JOBSHEET 10

PERCOBAAN 1

- **Kode Program Queue15**

```
public class Queue15 {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue15 (int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }
    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }
    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
    }
    public void peek() {
        if (!IsEmpty()) {
            System.out.println("Elemen terdepan: " + data[front]);
        } else {
            System.out.println("Queue masih kosong");
        }
```

```
}
public void print() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.println(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}
public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}
public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}
public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
```

```
        } else {
            dt = data[front];
            size--;
            if (IsEmpty()) {
                front = rear = -1;
            } else {
                if (front == max - 1) {
                    front = 0;
                } else {
                    front++;
                }
            }
        }
        return dt;
    }
}
```

- **Kode Program QueueMain15**

```
import java.util.Scanner;

public class QueueMain15 {
    public static void menu() {
        System.out.println("Masukkan operasi yang diinginkan:");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("-----------------");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan kapasitas Queue: ");
        int n = sc.nextInt();
        Queue15 Q = new Queue15(n);
        int pilih;

        do {
            menu();
            pilih = sc.nextInt();
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
```

```
                int dataMasuk = sc.nextInt();
                Q.Enqueue(dataMasuk);
                break;
            case 2:
                int dataKeluar = Q.Dequeue();
                if (dataKeluar !=0) {
                    System.out.println("Data yang dikeluarkan: " + dataKeluar);
                    break;
                }
            case 3:
                Q.print();
                break;
            case 4:
                Q.peek();
                break;
            case 5:
                Q.clear();
                break;
        }
    }
    while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);
  }
}
```

- **Hasil Running**

```
Masukkan kapasitas Queue: 4
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
1
Masukkan data baru: 31
```

```
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
4
Elemen terdepan: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
0
```

Pertanyaan P1

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?
2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {
    rear = 0;
```

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {
    front = 0;
```

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?
5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

6. Tunjukkan potongan kode program yang merupakan queue overflow!
7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawaban P1

1. Nilai -1 digunakan sebagai penanda bahwa queue masih kosong dan belum memiliki data. Sementara itu, atribut size bernilai 0 karena memang belum ada elemen yang tersimpan di dalam queue tersebut.
2. Potongan kode berikut digunakan untuk membuat antrian bersifat melingkar (circular), sehingga setelah mencapai indeks terakhir dapat kembali ke awal.
3. Potongan kode berikut juga berfungsi sama, yaitu menjaga pergerakan front tetap dalam pola melingkar.
4. Pada method print, perulangan dimulai dari front karena posisi awal data tidak selalu di indeks 0, melainkan bisa bergeser akibat operasi queue.
5. Kode tersebut berfungsi untuk menggeser indeks secara melingkar agar tidak keluar dari batas array.
6. Potongan kode queue overflow

```
public boolean IsFull() {
    if (size == max) {
        return true;
```

7. Modifikasi Enqueue

   Sebelum:

```
public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println(x: "Queue sudah penuh");
```

   Sesudah

```
public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println(x: "Queue sudah penuh");
        System.exit(status: 0);
```

   Modifikasi Dequeue

   Sebelum:

```
public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println(x: "Queue masih kosong");
```

   Sesudah:

```
public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println(x: "Queue masih kosong");
        System.exit(status: 0);
```

Hasil Running

```
Masukkan kapasitas Queue: 2
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
1
Masukkan data baru: 13
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
1
Masukkan data baru: 14
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----------------
1
Masukkan data baru: 15
Queue sudah penuh
```

PERCOBAAN 2

- **Kode Program MahasiswaQ15**

```
public class MahasiswaQ15 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public MahasiswaQ15(String nim, String nama, String prodi, String kelas){
        this.nim = nim;
        this.nama = nama;
```

```
        this.prodi = prodi;
        this.kelas = kelas;
    }
    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
}
```

- **Kode Program AntrianLayanan15**

```
public class AntrianLayanan15 {
    MahasiswaQ15[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan15(int max) {
        this.max = max;
        this.data = new MahasiswaQ15[max];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }
    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }
    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
    }
    public void peek() {
        if (!IsEmpty()) {
            System.out.println("Elemen terdepan: " + data[front]);
        } else {
            System.out.println("Queue masih kosong");
        }
    }
```

```
public void print() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.println(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}
public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}
public void Enqueue(MahasiswaQ15 dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
        System.exit(0);
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}
public MahasiswaQ15 Dequeue() {
    MahasiswaQ15 dt = null;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
```

```
        System.exit(0);
    } else {
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
            } else {
                front++;
            }
        }
    }
    return dt;
}
public void tambahAntrian(MahasiswaQ15 mhs) {
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambah mahasiswa.");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + " berhasil masuk ke antrian.");
}
public MahasiswaQ15 layaniMahasiswa() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return null;
    }
    MahasiswaQ15 mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}
public void lihatTerdepan() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
    } else {
        System.out.print("Mahasiswa terdepan: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[front].tampilkanData();
    }
```

```
    }
    public void tampilkanSemua() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Daftar Mahasiswa dalam Antrian:");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        for (int i = 0; i < size; i++) {
            int index = (front + i) % max;
            System.out.print((i + 1) + ". ");
            data[index].tampilkanData();
        }
    }
    public int getJumlahAntrian() {
        return size;
    }
}
```

- **Kode Program LayananAkademikSIAKAD15**

```
import java.util.Scanner;

public class LayananAkademikSIAKAD15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan15 antrian = new AntrianLayanan15(5);
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Akademik ===");
            System.out.println("1. Tambah Mahasiswa ke Antrian");
            System.out.println("2. Layani Mahasiswa");
            System.out.println("3. Lihat Mahasiswa Terdepan");
            System.out.println("4. Lihat Semua Antrian");
            System.out.println("5. Jumlah Mahasiswa dalam Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = sc.nextInt(); sc.nextLine();
            switch (pilihan) {
                case 1:
                    System.out.print("NIM   : ");
                    String nim = sc.nextLine();
                    System.out.print("Nama  : ");
                    String nama = sc.nextLine();
```

```
                System.out.print("Prodi : ");
                String prodi = sc.nextLine();
                System.out.print("Kelas : ");
                String kelas = sc.nextLine();
                MahasiswaQ15 mhs = new MahasiswaQ15(nim, nama, prodi, kelas);
                antrian.tambahAntrian(mhs);
                break;
            case 2:
                MahasiswaQ15 dilayani = antrian.layaniMahasiswa();
                if (dilayani != null) {
                    System.out.print("Melayani mahasiswa: ");
                    dilayani.tampilkanData();
                }
                break;
            case 3:
                antrian.lihatTerdepan();
                break;
            case 4:
                antrian.tampilkanSemua();
                break;
            case 5:
                System.out.println("Jumlah dalam antrian: " + antrian.getJumlahAntrian());
                break;
            case 0:
                System.out.println("Terima Kasih.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan !=0);
        sc.close();
    }
}
```

- **Hasil Running**

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 123
```

```
Nama  : Aldi
Prodi : TI
Kelas : 1G
Aldi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 124
Nama  : Bobi
Prodi : TI
Kelas : 1G
Bobi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1G
2. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Melayani mahasiswa: 123 - Aldi - TI - 1G

=== Menu Antrian Layanan Akademik ===
```

```
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 0
Terima Kasih.
```

Pertanyaan P2

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

Jawaban P2

- **Kode Program Modifikasi AntrianLayanan15**

```
public void lihatAkhir() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
```

```
    } else {
        System.out.println("Mahasiswa paling belakang: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[rear].tampilkanData();
    }
}
```

- **Kode Program Modifikasi LayananMahasiswaSIAKD15**

```
System.out.println("6. Cek Antrian Paling Belakang");


 case 6:
        antrian.lihatAkhir();
        break;
```

- **Hasil Running**

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu: 6
Mahasiswa paling belakang:
NIM - NAMA - PRODI - KELAS
124 - Bobi - TI - 1G
```

TUGAS

Gambar Diagram Class

Mahasiswa
nim: String nama: String prodi: String kelas: String
Mahasiswa(nim, nama, prodi, kelas) tampilData(): void

KRS
data: Mahasiswa[] front: int rear: int size: int max: int = 10 jumlahProses: int
isEmpty(): boolean isFull(): boolean clear(): void enqueue(): void dequeue(): void tampilSemua(): void tampilDepan2(): void tampilAkhir(): void info(): void

- **Kode Program MahasiswaT15**

```
public class MahasiswaT15 {
    String nim;
    String nama;
    String prodi;
    String kelas;
```

```
    public MahasiswaT15(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }
    public void tampilData() {
        System.out.println(nim + " | " + nama + " | " + prodi + " | " + kelas);
    }
}
```

- **Kode Program AntrianKRS15**

```
public class AntrianKRS15 {
    MahasiswaT15[] data;
    int front, rear, size;
    int max = 10;
    int jumlahProses = 0;

    public AntrianKRS15() {
        data = new MahasiswaT15[max];
        front = rear = -1;
        size = 0;
    }
    public boolean isEmpty() {
        return size == 0;
    }
    public boolean isFull() {
        return size == max;
    }
    public void clear() {
        front = rear = -1;
        size = 0;
        System.out.println("Antrian dikosongkan.");
    }
    public void enqueue(MahasiswaT15 m) {
        if (isFull()) {
            System.out.println("Antrian penuh!");
            return;
        }
        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % max;
        }
```

```
    data[rear] = m;
    size++;
    System.out.println("Berhasil masuk antrian.");
}
public void dequeue2() {
    if (size < 2) {
        System.out.println("Antrian kurang dari 2!");
        return;
    }
    System.out.println("Memproses 2 mahasiswa:");
    for (int i = 0; i < 2; i++) {
        data[front].tampilData();
        front = (front + 1) % max;
        size--;
        jumlahProses++;
    }
}
public void tampilSemua() {
    if (isEmpty()) {
        System.out.println("Antrian kosong.");
        return;
    }
    int i = front;
    for (int j = 0; j < size; j++) {
        data[i].tampilData();
        i = (i + 1) % max;
    }
}
public void tampilDepan2() {
    if (size < 2) {
        System.out.println("Kurang dari 2 antrian.");
        return;
    }
    data[front].tampilData();
    data[(front + 1) % max].tampilData();
}
public void tampilAkhir() {
    if (isEmpty()) {
        System.out.println("Antrian kosong.");
        return;
    }
    data[rear].tampilData();
}
public void info() {
```

```
        System.out.println("Jumlah antrian: " + size);
        System.out.println("Sudah proses: " + jumlahProses);
        System.out.println("Belum proses (maks 30): " + (30 - jumlahProses));
    }
}
```

- **Kode Program KRSMain15**

```
import java.util.Scanner;
public class KRSMain15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianKRS15 antrian = new AntrianKRS15();

        int pilih;
        do {
            System.out.println("\n=== MENU ANTRIAN KRS ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Proses KRS (2 Mahasiswa)");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Info Antrian");
            System.out.println("5. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("NIM   : ");
                    String nim = sc.nextLine();
                    System.out.print("Nama  : ");
                    String nama = sc.nextLine();
                    System.out.print("Prodi : ");
                    String prodi = sc.nextLine();
                    System.out.print("Kelas : ");
                    String kelas = sc.nextLine();
                    antrian.enqueue(new MahasiswaT15(nim, nama, prodi, kelas));
                    break;
                case 2:
                    antrian.dequeue2();
                    break;
                case 3:
                    System.out.println("Semua antrian:");
                    antrian.tampilSemua();
```

```
                System.out.println("\n2 terdepan:");
                antrian.tampilDepan2();
                System.out.println("\nTerakhir:");
                antrian.tampilAkhir();
                break;
            case 4:
                antrian.info();
                break;
            case 5:
                antrian.clear();
                break;
        }
    } while (pilih != 0);
  }
}
```

- **Hasil Running**

```
=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 123
Nama  : Aldi
Prodi : TI
Kelas : 1G
Berhasil masuk antrian.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
```

```
0. Keluar
Pilih: 1
NIM   : 124
Nama  : Bobi
Prodi : TI
Kelas : 1G
Berhasil masuk antrian.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 125
Nama  : Elsa
Prodi : TI
Kelas : 1G
Berhasil masuk antrian.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 126
Nama  : Wita
Prodi : TI
```

```
Kelas : 1G
Berhasil masuk antrian.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 127
Nama  : Rara
Prodi : TI
Kelas : 1G
Berhasil masuk antrian.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 2
Memproses 2 mahasiswa:
123 | Aldi | TI | 1G
124 | Bobi | TI | 1G

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
```

```
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 3
Semua antrian:
125 | Elsa | TI | 1G
126 | Wita | TI | 1G
127 | Rara | TI | 1G

2 terdepan:
125 | Elsa | TI | 1G
126 | Wita | TI | 1G

Terakhir:
127 | Rara | TI | 1G

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 4
Jumlah antrian: 3
Sudah proses: 2
Belum proses (maks 30): 28

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
```

```
5. Kosongkan Antrian
0. Keluar
Pilih: 5
Antrian dikosongkan.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 3
Semua antrian:
Antrian kosong.

2 terdepan:
Kurang dari 2 antrian.

Terakhir:
Antrian kosong.

=== MENU ANTRIAN KRS ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Antrian
4. Info Antrian
5. Kosongkan Antrian
0. Keluar
Pilih: 0
```